Fuzzing the Solidity Compiler

Bhargava Shastry
Ethereum Foundation

@ibags

bshastry

whoami

- Security engineer, Solidity team
- Semantic testing of Solidity compiler

Find security-critical bugs in the compiler before it is shipped

tl;dr:

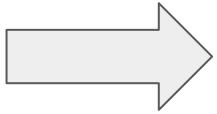
- Threat model: Incorrect code generation
- Randomly generated valid Solidity (yul) programs test compiler
- Found 2 security relevant bugs in EVM optimizer
  - Low or very low security impact
- Found 5 other bugs in experimental optimizer
- Continuous fuzzing for early bug discovery

Introduction

Threat model

- Compiler user (developer) is not malicious
- Bugs introduced by the optimizer

```
function foo() -> x {
  x := 2
}
mstore(0, foo())
```



```
mstore(0, 2)
```

Fuzz testing in a nutshell

```
while not ctrl + c
do
   input=gen_input()
   runProgram(input)
done
```

Limitation of random fuzzing

```
contract C {
  function foo()
public {

do_something();
  }
}
```

Mutation

```
contract C {
  fu#!3ion foo()
puX^&c {

do_something();
  }
}
```

Accepted by parser

Rejected by parser

Fuzzing a compiler requires generating valid programs...

... generating a valid program requires structure awareness

Approach

Write a specification

Specification written in protobuf language

```
message Block {
  repeated Statement stmts;
}
...
message program {
  repeated Block blocks;
}
```

Full spec:
https://github.com/ethereum/solidity/blob/develop/test/tools/ossfuzz/yulProto.proto

Input generation

- Input generated and mutated by libprotobuf-mutator
- Each input is a tree

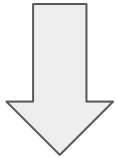
```
blocks { stmts { ifstmt { condition {
binaryOp { eq { op1: varref{id: 0} op2: 0}
} } } } }
```

Input conversion

- Converter is source-to-source translator
- Input: protobuf serialization format
- Output: yul program

Example

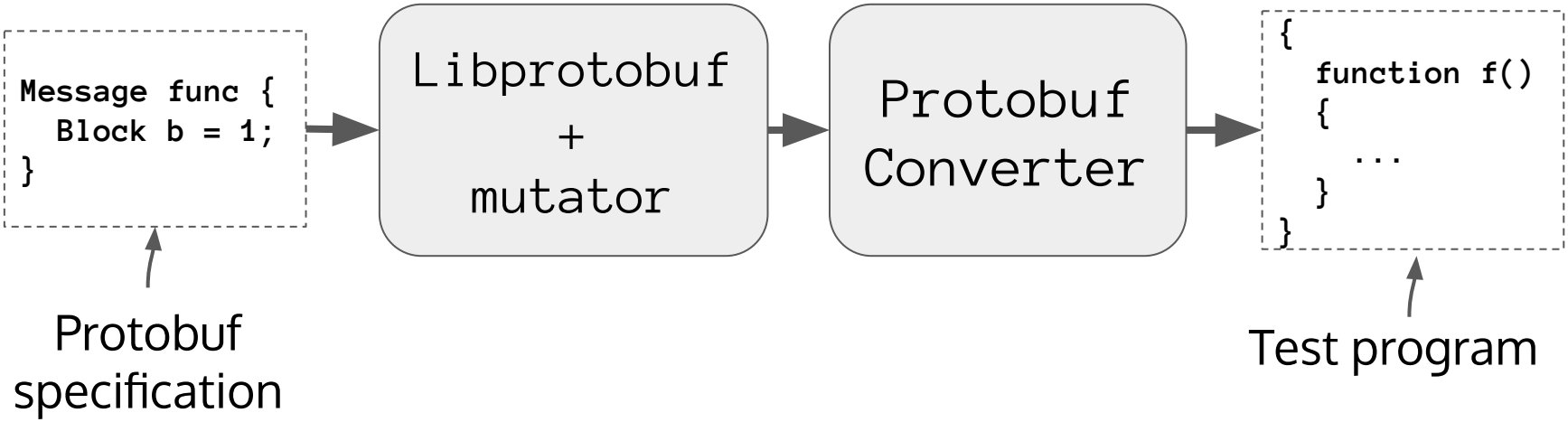
```
blocks { stmts { ifstmt { condition {
binaryOp { eq { op1: varref{id: 0} op2: 0}
} } } } }
```



Conversion

```
if x_0 == 0
```

Test program generation



Correctness testing requires encoding expectation somehow

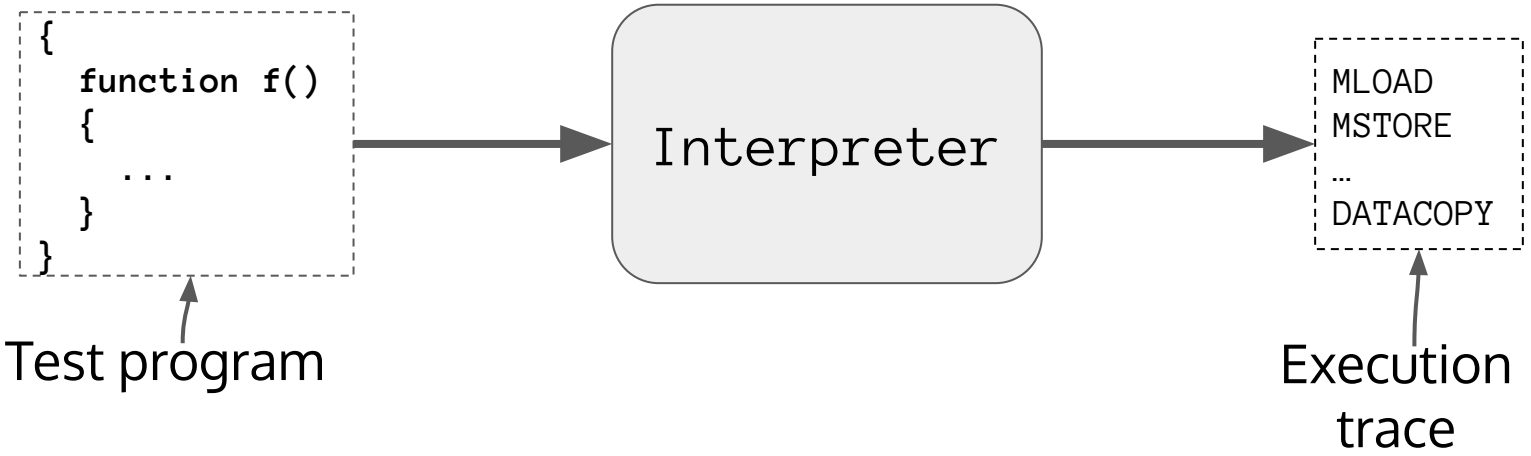
Differential fuzzing

- Track side-effects of execution
- Run program
- Run optimized program
- Compare side-effects

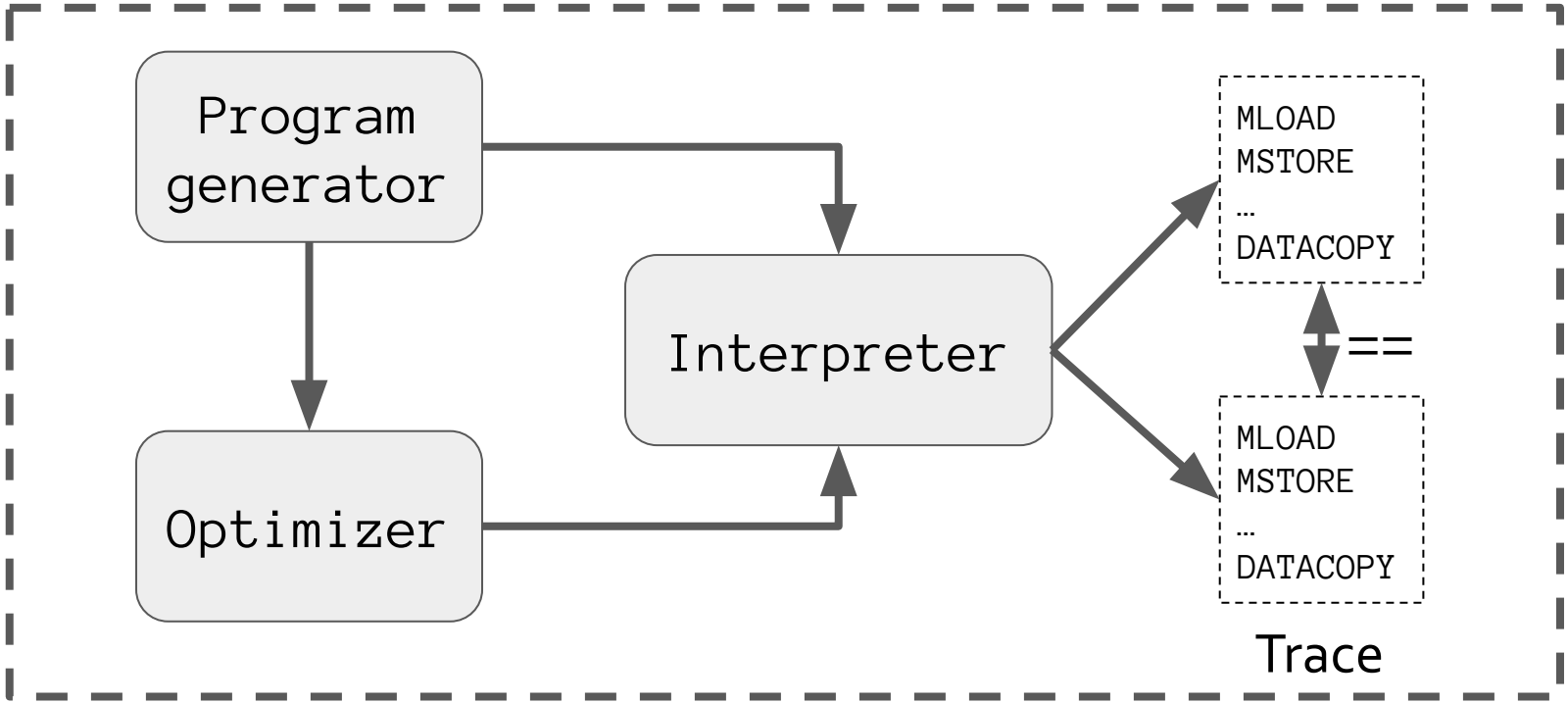
Yul interpreter

- Interprets arbitrary yul program
- Outputs side-effects as a trace (string)

Yul interpreter

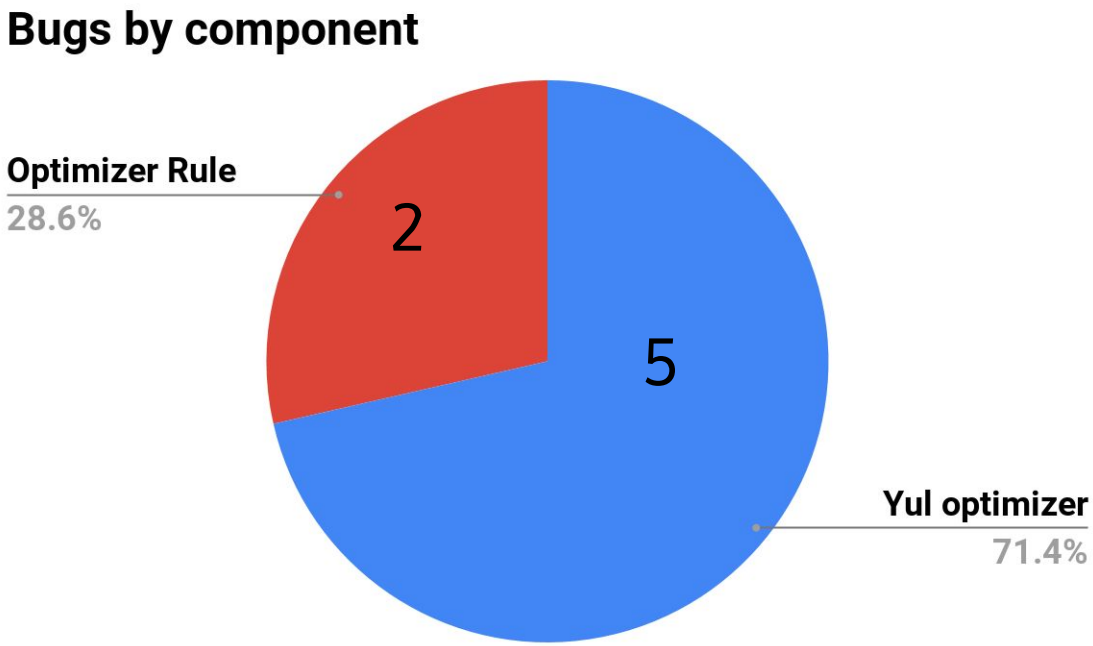


Fuzzing Setup

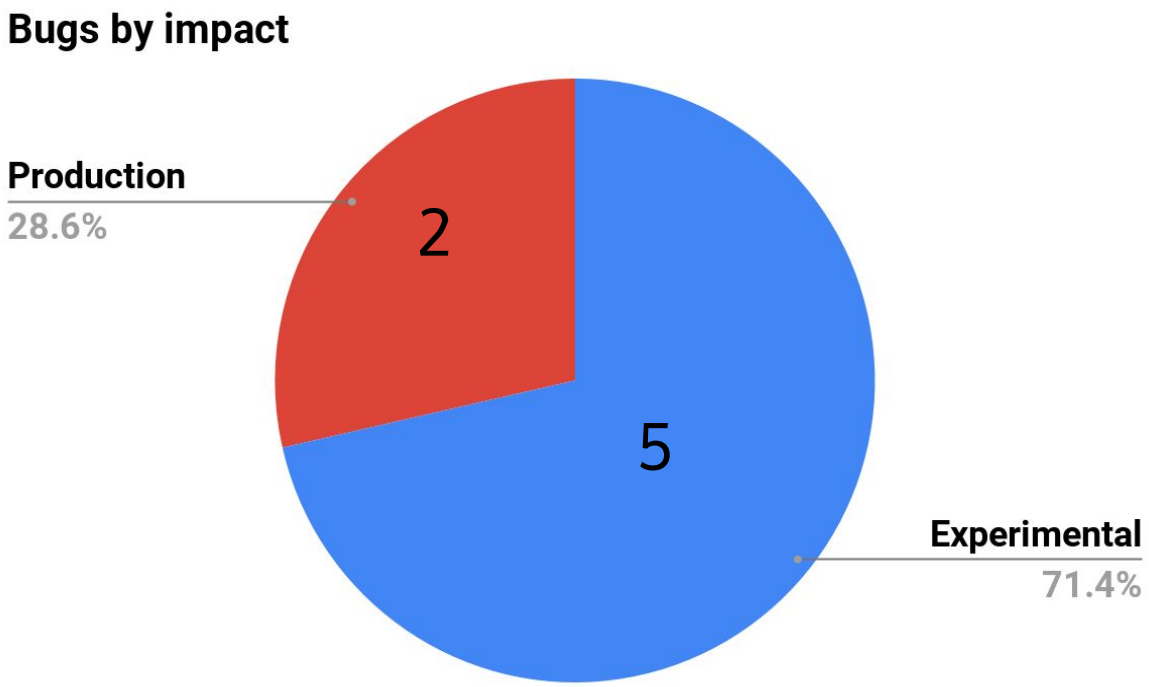


Results

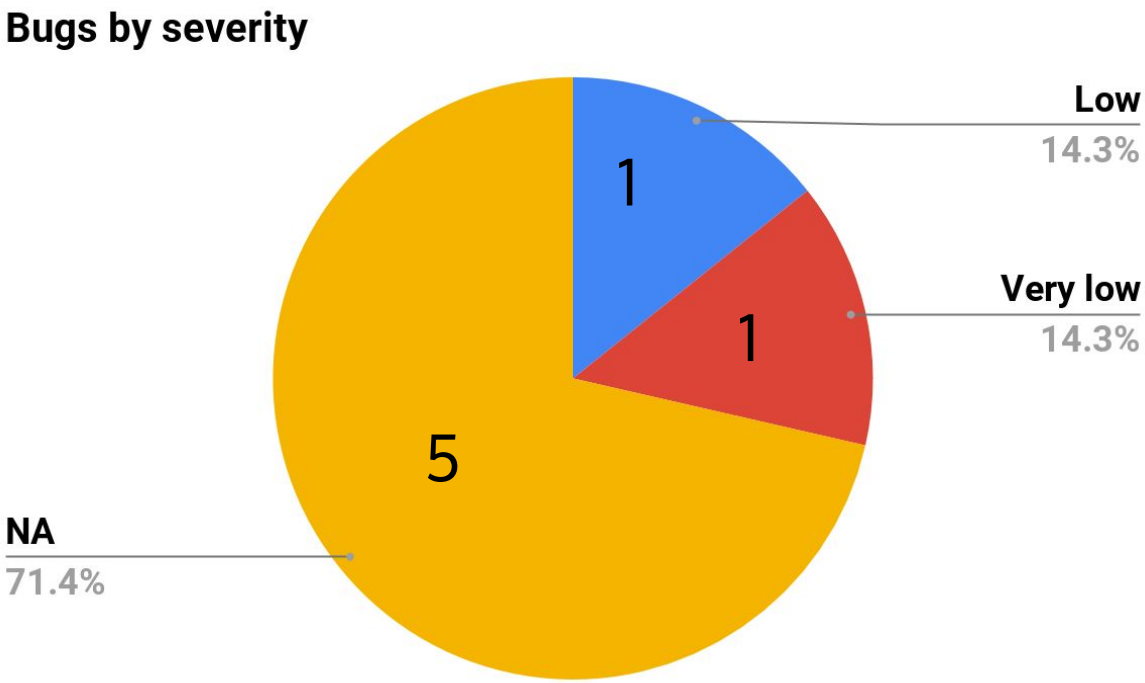
Bugs by component



Bugs by impact



Bugs by severity



Challenges

- Find high-severity bugs using fuzz testing
  - Slow test throughput (~1 test per second)
- Test Abiv2encoder
    - Generate test program (Reasonably fast)
    - Compile program (Slowest)
    - Run program on EVM (Slow)
    - Assert output validity (Very fast)

Conclusion

Conclusion

- Continuous structure-aware fuzzing for early bug discovery
- Useful for testing optimizer and data en/decoding
- Decent assurance
  - Evidence that it works
  - No formal guarantees though

Thank you!

https://github.com/ethereum/solidity

A tester's vain attempt to make their bug stand out in the next bug triage meeting
Come on little bug... you can do it!!
cartoontester.blogspot.com © 2013

Extras

Detectability

- Security relevant bugs
  - Triggered by constant expression e.g., byte(x, 31), shl(not(0), shl(1, x)
  - Regex to gauge impact

Early bug discovery

We are beginning to find bugs before a release!

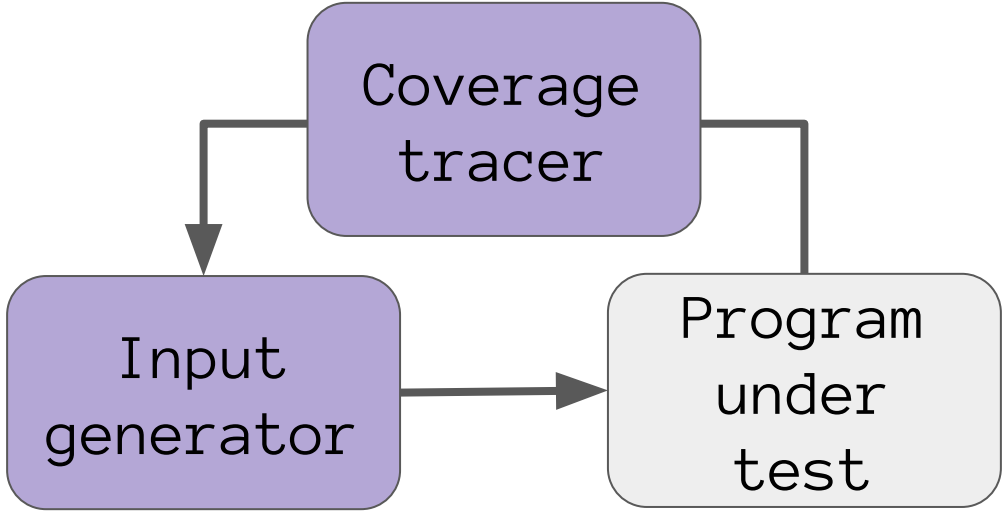
X: Does this bug require a Changelog entry?

Y: No, because the bug is in an unreleased feature!

Continuous fuzzing

- Every day, Google's oss-fuzz
  - Builds latest develop code
  - Fuzzes it
  - Minimizes test case
- Bug reports filed automatically
  - Sent to Inbox!

Fuzz testing



Fuzzing toolchain

`coverage_bitmap={0}`

- Global view of test coverage

`input_queue={seeds}`

- Test inputs

Instrumentation

```
$ clang -fsanitize=fuzzer myProgram.cpp
```

- Instrument program to guide fuzzing

Fuzzing oracle

```
bool is_input_interesting()
```

- Toolchain (clang/llvm) provides oracle

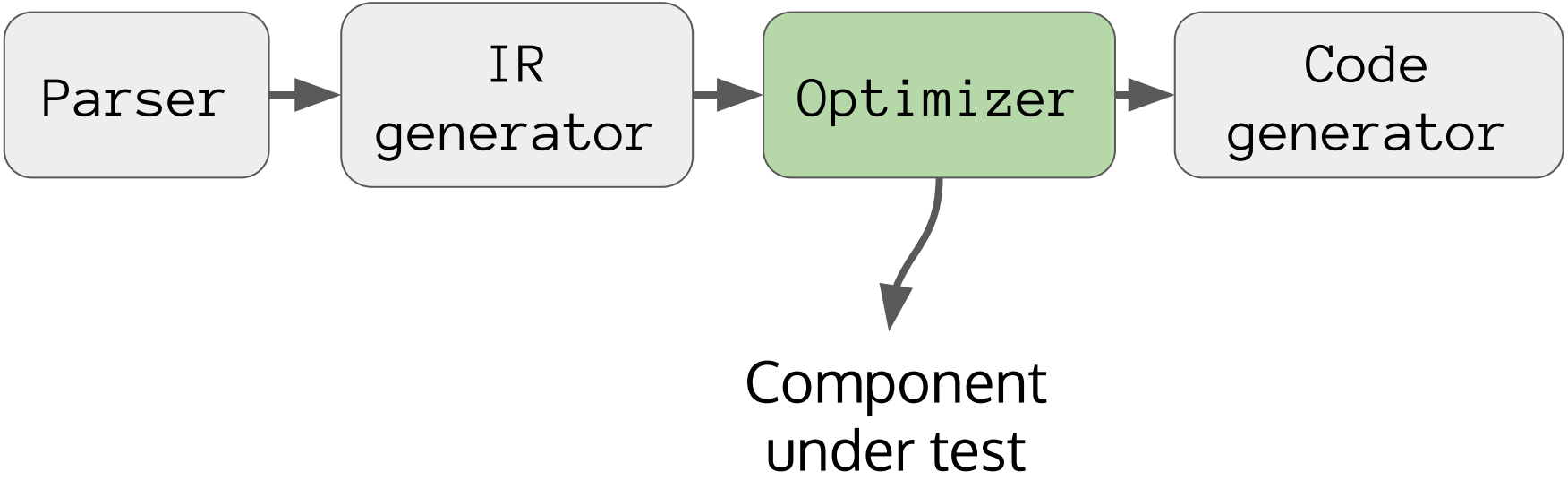
Fuzz testing in practice

```
while not ctrl + c
do
  input=mutate(pick_input(input_queue))
  coverage = runInstrumentedProgram(input)
  if (is_input_interesting())
    input_queue.add(input)
done
```

Background

Solidity Compiler Architecture

In-depth talk: Yul, eWasm, Solidity: Progress and Future Plans



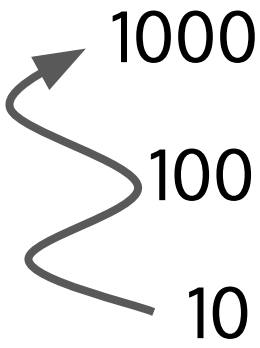
Optimizer rule

```
SHL(B, SHL(A, X)) -> SHL(min(A+B, 256), X)
```

- Double shift is additive
- Maximum shift possible is 256* bits
- Question: Does the rule express intent in all cases?

* EIP 145 defines shift by 256 bits to be zero

```
SHL(1, SHL(1, 2)) = SHL(2, 2) = 8
```



Corner case (fuzzer generated)

```
{
  foo(2)
  function foo(x)
  {
    mstore(0, shl(1,shl(not(0),x)))
  }
 }
```

u256 arithmetic

```
not(0) = 0xffffffffffffff...ffffffffffffffffff
```

256 bits

```
not(0) + 1 = 0
```

Yul optimization rule

```
SHL(B, SHL(A, X)) -> SHL(min(A+B, 256), X)
```

- min(A+B, 256) = 0
- Optimized code: SHL(0, X) = X!
- Expected output: SHL(256, X) = 0

Step Through

```
{
  foo(2)
  function foo(x)
  {
    mstore(0, shl(1,shl(not(0),x)))
  }
 }
```

- Actual output: 2
- Expected output: 0

Related Work

- Csmith [1]
- Prog-fuzz [2]

[1] https://embed.cs.utah.edu/csmith/

[2] http://www.vegardno.net/2018/06/compiler-fuzzing.html

Related Work

- Csmith [1]
- Prog-fuzz [2]

[1] https://embed.cs.utah.edu/csmith/

[2] http://www.vegardno.net/2018/06/compiler-fuzzing.html

Test harness

```
void DEFINE_PROTO_FUZZER(Proto& input)
{
    string testProgram = protoConverter(input);
    string unoptimizedTrace = interpret(testProgram);
    string optProgram = optimize(testProgram);
    string optimizedTrace = interpret(optProgram);
    assert(unoptimizedTrace == optimizedTrace);
}
```